

Diagnostic Reports

Overview

This document describes the Diagnostic Reports feature.

The goal of Diagnostic Reports

It provides a self-service approach for developers to diagnose production performance issues in the Rails monolith.

The unique ability of this feature is to write files with diagnostic data and upload them to GCS. There are no limitations on the data format (it could be unstructured). It allows uploading large reports (potentially hundreds of MB). This is what makes it different from using Prometheus or Kibana.

Target environment

The feature is only available on SaaS.

This is not an application feature. That would be a stretch goal at best.

The reason this isn't particularly useful for anyone except us is that both production engineers (GitLab) and self-managed admins can just shell into a box and pull a heap dump. We cannot.

How to access reports / GCS

Access to the buckets would need to be fulfilled via an Access Request

The buckets are:

- `gitlab-diagnostic-reports-gstg` in the `gitlab-staging-1` GCP project
- `gitlab-diagnostic-reports-gprd` in the `gitlab-production` GCP project

For example, to get a list of staging reports:

```
gsutil ls -p gitlab-staging-1 'gs://gitlab-diagnostic-reports-gstg/heap_dump.2023-11-09*'
```

To download a report:

```
gsutil cp gs://gitlab-diagnostic-reports-gstg/heap_dump.2023-11-09.09:26:27:876.puma_3.57fe
```

We configured a GCS Lifecycle rule to delete all report files after a successful upload to save storage costs. Refer to this MR for more details.

Development setup While working on this feature, you may want to run the uploader and push data to an actual GCS destination.

For this you must perform the following steps:

1. Create a GCP sandbox via <https://about.gitlab.com/handbook/infrastructure-standards/realms/sandbox/>
 - Note the project ID; set `GITLAB_DIAGNOSTIC_REPORTS_PROJECT` to this.
1. Create a new GCS bucket to hold the reports. Note that GCS buckets must be *globally unique*, so choose wisely.
 - Note the bucket name; set `GITLAB_DIAGNOSTIC_REPORTS_BUCKET` to this.
1. Create a service account with the `Storage Object Creator` role applied and create and download a key file.
 - Note the path to this file; set `GITLAB_GCP_KEY_PATH` to this.
1. Set `GITLAB_DIAGNOSTIC_REPORTS_PATH` to wherever reports are written to.

In an environment with these variables set, run:

```
bin/rails r bin/diagnostic-reports-uploader
```

By default, logs are written here:

```
tail -f log/diagnostic_reports_json.log
```

Production setup In production, we run Puma and Sidekiq in containers. To upload diagnostic data written by these systems, we mount a shared volume and run the uploader script in a sidecar container.

Functionality (high-level overview)

Currently, we have a single type of report: Jemalloc stats. Jemalloc is the memory allocator we use for `gitlab-rails`.

- We pull memory allocator stats with the configured frequency and dump them to disk. We call these dumps “reports” (there could be more reports, such as heap dumps).
- We upload this report to the GCS bucket for future analysis.

Functionality (implementation details)

The system consists of two major parts:

- Reports Producer
- Reports Uploader

Reports Producer

- Reports are produced only if `GITLAB_DIAGNOSTIC_REPORTS_ENABLED` env variable is set.
- There are report implementations per every kind of report.

- On startup, every Puma worker spawns a daemon responsible for executing reports.
- The daemon runs with the configured frequency.
- We add a jitter to avoid all workers serving the report at the same time which could cause availability issues.
- Each report is configured with the path where it should store the report so the uploader could pick it.

Reports Uploader

- All webservice pods run a dedicated side car container.
- The purpose is to scan the target report directory and upload all its contents into the configured GCS bucket.
- No matter if the upload failed or succeeded, the report file is deleted to free up the space.

Adding additional report

Things to consider

- Currently, Reports Producer only works from Puma workers. It does not work from Sidekiq yet. Refer to this issue.
- First, decide if the Diagnostic Reports framework is the best solution. If you send small amounts of structured data, consider using GitLab logs. If you can present your data as metrics, check Prometheus Metrics. A good scenario for Diagnostic Reports is when you need to send a large amount of data produced by the Ruby process, which will bloat the log record. Another potential scenario is sending poorly structured data to the GCS. The data will be parsed and analyzed by the consumer (having GCS access) asynchronously.
- Our reporter system is a timer-based or event-based trigger, which is executed in the Puma worker thread. Be aware and measure the potential performance impact which could cause availability issues.
- Create a separate `:ops` feature flag for a new report
- Note that even if the report is not produced by a Puma worker, you could still use the capabilities of the Reports Uploader to pick up and upload the file to GCS from the target directory.

Performance impact and availability concerns

- If you add a new Reports producer, you need to be aware that this code will be executed by the Puma worker process, which means that it will compete for resources with the functionality that actually serves the request. Because of that, you should be very careful to avoid availability issues. Refer to the MR to learn how we verified that the availability impact of the Jemalloc report is minimal

- Note that we have limited space in the container volume set for reports: refer to the k8s-workloads for exact values.
- We use an emptyDir with a size cap to control the growth of said directory and to prevent our nodes from running out of disk space.
- It means that overflowing this amount will trigger container restart which you should avoid at all costs because this is a Puma webserver container.
- Consider tweaking the dir size if you adding a new report
- The configured volume will be wiped after the container restart (e.g. redeploy).
- It is also possible to tweak the uploader frequency because it will clean up the files it uploads.

Troubleshooting and monitoring

How to disable the feature

- Disable the reporter completely by unsetting the `GITLAB_DIAGNOSTIC_REPORTS_ENABLED` env var. Check this MR for reference.
- You can also disable each report separately with its own `:ops` feature flag. Use `report_jemalloc_stats` for Jemalloc stats report. Do this if you want to troubleshoot the target report while keeping other reports running.

Logs and metrics

- We log report producer events. Filter with `json.perf_report : *`. For example, on `gprd`: [link](#)
- We log uploader events. Filter with `json.subcomponent:diagnostics-uploader`. For example, on `gprd`: [link](#)
- We also update Prometheus counter `gitlab_diag_report_duration_seconds_total`, check it [here](#)